

WEB SISTEMAK

2025/2026 IKASTURTEA

HTTP - HyperText Transfer Protocol

Konpresioa. Adibidea Python-ekin

Erantzunaren edukiaren kodifikazioa. Adibidea Wireshark-ekin



Web Sistemak by [Oskar Casquero](#) & [María Luz Álvarez](#) is licensed under a [Creative Commons Reconocimiento 4.0 Internacional License](#).

HTTP-REN FUNTZIONAMENDUA: KONPRESIOA

- **Zelan laburtzen da erantzunen tamaina?**

- Orokorrean, HTTP erantzun baten edukia testua da: HTML, XML, JSON, CSS.
- Testuak konpresioarako erantzun ona aurkezten du.
- HTTP protokoloak mezuaren gorputza konprimatzeko aukera ematen du, horretarako algoritmorik erabiliena *gzip* delarik.
- Konpresio algoritmoa bi goibururen bitartez adierazten da:
 - Eskieran: “Accept-Encoding”
 - Bezeroak konprimatutako eduki bat deskonprimatzeko gaitasunik ez badu, goiburu honek “identity” balioa hartuko du.
 - Erantzunean: “Content-Encoding”
 - Mezuaren gorputza konprimatu gabe bidaltzen bada, ez da goiburu hau adierazten.

HTTP-REN FUNTZIONAMENDUA: KONPRESIOA

- Demagun eduki konprimatuekin lan egiteko gai den bezero batek ondorengo URI-a duen baliabidea eskatzen duela: <https://www.google.es/>

Eskaera baten adibidea

```
GET / HTTP/1.1
Host: www.google.es
Accept: text/html
Accept-Encoding: gzip,identity;q=0.5
Accept-Language: en-US,es-ES;q=0.8
User-Agent: Mozilla Windows Desktop
```

Erantzun baten adibidea

```
HTTP/1.1 200 OK
Date: Mon, 30 Nov 2015 21:03:14 GMT
Content-Encoding: gzip
Content-Length: 5543
Content-Type: text/html; charset=ISO-8859-1
```

KONPRIMATUTAKO EDUKIA



```
8Ó|ñ!â¥ÊîF@eL!_N6|æYÁ|_À?e'S|Ö■_Q_1_1_ÆF±E|_#'^|
|_#'^|
8Ó_T0üs°L!8UâS y DX!|B+-i5]2Äü":â%Ç_Tô¶e*M-:ÓA(=ô!|Ä²X#çL_d||8±`
¹@70Ç ä      ²²ád0Tê' LH?NÇx2VâÉ£i4B°ç-âMÇæ²Ai2É-üy0a:8îâÇAI
(â²)²eDr*-P!±ú=E·äc²¶üÐÉ_TÇi||c 4$F|â||ô:h'1Séu'=ÇÖbätüüYüB||
```

ADIBIDEA: KONPRESIOA

```
import requests
import sys
import zlib

metodoa = 'GET'
uria = "https://www.google.es/"
goiburuak = {'Host': 'www.google.es'}

compressed = False
if len(sys.argv) == 1:
    goiburuak['Accept-Encoding'] = 'identity'
elif sys.argv[1] == 'compress':
    compressed = True
    goiburuak['Accept-Encoding'] = 'gzip'

erantzuna = requests.request(metodoa, uria, headers=goiburuak, allow_redirects=False, stream=True)

kodea = erantzuna.status_code
deskribapena = erantzuna.reason
print(str(kodea) + " " + deskribapena)
for goiburua in erantzuna.headers:
    print(goiburua + ": " + erantzuna.headers[goiburua])

print("RESPONSE CONTENT LENGTH: " + str(len(erantzuna.raw.data)) + " byte")
if compressed:
    edukia compressed = erantzuna.raw.data
    edukia uncompressed = zlib.decompress(edukia compressed, 16+zlib.MAX_WBITS)
    print("UNCOMPRESSED RESPONSE CONTENT LENGTH: " + str(len(edukia_uncompressed)) + " byte")
```

The diagram consists of two red arrows. The first arrow starts from the `stream=True` parameter in the `requests.request` call and points to a red box containing `erantzuna.raw.data`. The second arrow starts from this box and points to the `edukia compressed` variable in the decompression logic.

- requests liburutegiak *gzip* eta *deflate* konpresio formatuak automatikoki deskonprimatzen ditu.
- Erantzunaren edukira atzipen zuzena nahi badugu, eskaeran *stream=True* parametroa sartu eta erantzunean *raw.data* eremua irakurri behar dugu

ADIBIDEA: KONPRESIOA

```
(venv) C:\Users\cvzcaio\Dropbox (Personal)\docencia\grado\Sistemas Web\SW 2020\Praktikal>python compression.py compress
200 OK
Date: Wed, 12 Feb 2020 07:39:19 GMT
Expires: -1
Cache-Control: private, max-age=0
Content-Type: text/html; charset=ISO-8859-1
P3P: CP="This is not a P3P policy! See g.co/p3phelp for more info."
Content-Encoding: gzip
Server: gws
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN
Set-Cookie: 1P_JAR=2020-02-12-07; expires=Fri, 13-Mar-2020 07:39:19 GMT; path=/; domain=.google.es; Secure, NID=197=TuCqh
sRxyTUWCZOVawM6ScZOBD0; expires=Thu, 13-Aug-2020 07:39:19 GMT; path=/; domain=.google.es; HttpOnly
Alt-Svc: quic=":443"; ma=2592000; v="46,43",h3-Q050=":443"; ma=2592000,h3-Q049=":443"; ma=2592000,h3-Q048=":443"; ma=2592000
Transfer-Encoding: chunked
RESPONSE CONTENT LENGTH: 5584 byte
UNCOMPRESSED RESPONSE CONTENT LENGTH: 13116 byte
```

- Konprimatutako edukiaren luzera: 5584 zortzikote
- Konprimatutako gabeko edukiaren luzera: 13116 zortzikote
% 57ko aurrezpena mezuaren gorputzaren tamainan

HTTP-REN FUNTZIONAMENDUA:

ERANTZUNAREN EDUKIAREN KODIFIKAZIOA

- Mezuaren gorputza zortzikotez (byte-z) osoturik dago:
 - Testua (testu sinplea, HTML, CSS, XML, JSON, CSS, ...)
 - Eduki binarioa (PDF, bideoa, exekutagarriak, etc.)
- Eduki binarioak aplikazio edo plataforma jakin batzuentzako pentsaturik daude. Adibidez, PDF-ak PDF-irakurle batek soilik ulertu ditzake ondo. PDF bat testu editore arrunt batekin zabaldu edo irteera estandarrera bidaltzen bada, bere edukiaren adierazpen oker bat bistaratzen da, testu oinarria duena.
- Testua programa ugarirekin bistaratu daiteke, baina guztiek ez dute zortzikoteak dekodifikatzeko konfigurazio berdina erabiltzen.
 - Adibidez, jatorrian fitxategi bat [UTF-8](#) erabiliz kodifikatu bazan, "ñ" ikurra C3 B1 byte-ekin gordeko da.
 - Fitxategia [ISO-8859-1](#)-en konfiguratutako testu editore baten bitartez zabaltzen badugu, dekodifikazioaren eraginez ondorengo bi ikurrak bistaratuko dira: Ã±

KODIFIKAZIOARI BURUZKO ARGIBIDE BATZUK

Kode taula	Ikurra	Hex		binarioa															
US-ASCII (7 bit)	H	--	48															0	1 0 0 1 0 0 0
	ñ	--	--																
	á	--	--																
ISO-8869-1 (1 zortzikote)	H	--	48															0	1 0 0 1 0 0 0
	ñ	--	F1															1	1 1 1 0 0 0 1
	á	--	E1															1	1 1 0 0 0 0 1
UTF 8 (1 - 4 zortzikote)	H	--	48															0	1 0 0 1 0 0 0
	ñ	C3	B1	1	1	0	0	0	0	1	1	1	0	1	1	0	0	0	1
	á	C3	A1	1	1	0	0	0	0	1	1	1	0	1	0	0	0	0	1

- Demagun ondorengo testua kodifikatu nahi dela: Hola, Iñaki Vázquez!

US-ASCII: Ezin da kodifikatu, “ñ” y “á” ikurrak ez baitira US-ASCII hiztegien existitzen.

Latin-1: 48 6F 6C 61 2C 20 49 **F1** 61 6B 69 20 56 **E1** 7A 71 76 65 7A 21

UTF-8: 48 6F 6C 61 2C 20 49 **C3 B1** 61 6B 69 20 56 **C3 A1** 7A 71 76 65 7A 21

KODIFIKAZIOARI BURUZKO ARGIBIDE BATZUK

z

gorde
ASCII
latin-1
UTF-8

5A

zabaldu
ASCII
latin-1
UTF-8

z

gorde
ASCII

✗

ñ

gorde
latin-1

F1

zabaldu
ASCII
latin-1
UTF-8

✗

ñ

gorde
UTF-8

C3 B1

zabaldu
ASCII
latin-1
UTF-8

✗

ñ

Ã ±

HTTP-REN FUNTZIONAMENDUA:

ERANTZUNAREN EDUKIAREN KODIFIKAZIOA

- “Content-Type” goiburaren bitartez, zerbitzariak, itzultzen duen eduki mota adierazi dezake: text/html, application/pdf, ...
- Nabigatzaileak “Content-Type” goiburua irakurtzen duenean, bere balioa testu motakoa edo beste mota batekoa ote den egiaztatzen du.
 - Edukia beste mota batekoa bada, bera zabaltzeko plugin-ik konfiguratuta ote duen egiaztatzen du. Horrela ez bada, erabiltzaileari edukia gordetzeko edo dagokion aplikazioarekin zabaltzeko aukera ematen dio.
 - Edukia testu motakoa bada, nabigatzaileak berak interpretatu (bistaratu) egiten du. Horretarako zelan kodifikatuta dagoen jakin behar du.
 - HTTP prokoloak edukiaren kodifikazioa “Content-Type” goiburuan adierazteko aukera du:

Content-Type: text/html; charset=ISO-8859-1

ADIBIDEA:

KODIFIKAZIO ARAZO BATEN AZTERKETA

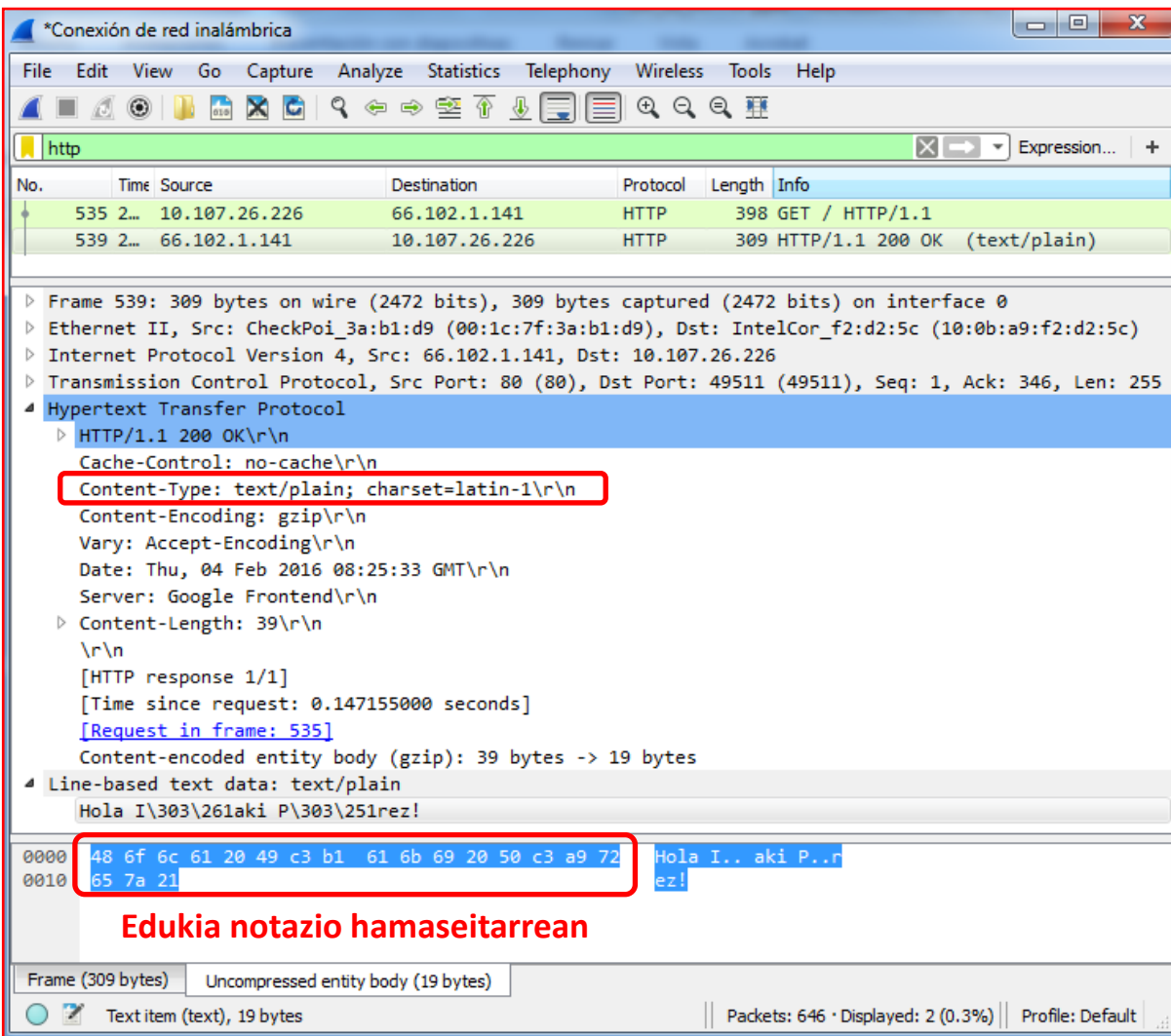
- Wireshark erabilita, nabigatzailean ondorengo URI-a zabaldu eta bere kodifikazio arazoaren arrazoia aurkitu

<http://ws-responsecontentcoding.appspot.com/>



ADIBIDEA:

KODIFIKAZIO ARAZO BATEN AZTERKETA



Wireshark packet capture showing an HTTP response. The packet list shows a GET request and a 200 OK response. The packet details show the response headers and body. The packet bytes pane shows the raw data of the response body, which is 'Hola I.. aki P..r ez!'.

Edukia notazio hamaseitarrean

- Nabigatzaileak edukia “text/plain” motakoa dela ikusten du, beraz, bera arduratzen edukia interpretatzeaz.
- Nola interpretatu behar du testua nabigatzaileak? Hau da, zein kodifikazio taula erabili behar du? “charset” parametroak adierazten duena; kasu honetan: latin-1.

48 → H
6F → o
6C → l
61 → a
20 →
49 → I
C3 → Ñ
B1 → ±

ADIBIDEA:

KODIFIKAZIO ARAZO BATEN AZTERKETA

- Zergaitik gertatzen da hau?
 - Zerbitzaria programatu duenak edukia latin-1-en kodifikatuta dagoela adierazten duen kode lerroa idatzi du:
- Baina ez du kontuan hartu *write* metodoa erabitzen denean zerbitzariak edukia UTF-8-n kodifikatzen duela:

```
self.response.charset = 'latin-1'
```

```
self.response.write("Hola Iñaki Pérez!")
```

Content-Type: text/plain; charset: latin-1

```
import webapp2class MainHandler(webapp2.RequestHandler):
    def get(self):
        self.response.content_type = 'text/plain'
        self.response.charset = 'latin-1'
        self.response.write("Hola Iñaki Pérez!")

app = webapp2.WSGIApplication([('/', MainHandler)], debug=True)
```